

AI 小说转剧本

当前项目主要问题说明

流式可靠性 · AI 成本 · 任务系统 · 前端性能

导读 这份说明解决什么

供团队同步、评审决策和后续拆分任务使用。

本项目已经完成小说正文、章节、故事资产、场景大纲、Scene 剧本、校验和 YAML 导出的主链路，但当前体验中的主要风险集中在流式预览、任务状态和模型上下文。问题并非单纯由网络或模型造成，代码中存在可复现的生命周期、超时和完成判断缺陷。

当前判断
首先修可靠性，再优化延迟和成本，最后处理多实例与规模化。数据库连接池和索引不是当前最优先的性能矛盾。

问题分级

级别	定义	数量	处理要求
P0	会导致断流、串流、假成功或资源失控	6 类	优先止血，完成专项测试
P1	显著增加延迟、模型费用、SQL 或前端卡顿	7 类	在路演稳定后集中优化
P2	演示规模不明显，扩展和多实例后会放大	5 类	形成明确技术债和方案

最需要团队统一的认识

- 流式预览目前不是正式 Scene 生成结果，而是一次额外的、不落库的模型调用。
- 前端切换 Scene 时旧流不会主动取消，确实存在内容串场风险。
- Scene SSE 的 120 秒超时小于 AI 默认 180 秒超时，存在确定性的超时竞争。
- 每个 Scene 都携带全项目实体和事件，模型上下文是当前最大的性能与成本优化点。
- RabbitMQ 当前默认全局串行，稳定但会让所有项目互相排队。

01 P0 : Scene 流生命周期失控

这是当前流式体验最直接的错误来源。

1.1 切换 Scene 后旧 EventSource 不关闭

现象	影响	代码证据	建议方向
EventSource 只在 done、failed 或 error 时关闭；Scene、项目切换和组件卸载没有清理。	旧 Scene chunk 可能继续写入当前面板；旧请求继续消耗模型和线程。	useWorkbench.ts:642-716、948-993	保存当前连接到 ref；切换和卸载时关闭；使用 stream token 校验 projectId/scenelId。

1.2 半截流被当作成功缓存

现象	影响	代码证据	建议方向
缓存只保存字符串，不区分 streaming、completed 和 failed；只要有一个 chunk 就会被复用。	断流后再次点击仍显示半截内容；重生成或重新分析后可能展示旧预览。	useWorkbench.ts:643-649、694-701	缓存增加状态和版本；仅收到有效 done 后复用；上游数据变化时失效。

可复现方式

开始 S001 流式预览后立即选择

S002；或在返回部分内容后断网并重新点击预览。观察内容是否串场、缓存是否仍被复用。

验收标准

- 连续切换 Scene 20 次不出现跨 Scene 内容。
- 切换项目和关闭页面后旧模型请求能及时终止。
- 失败缓存允许重新生成，Scene 更新后旧缓存自动失效。

02 P0：超时、线程与完成判断

当前配置可能在模型正常生成时主动截断浏览器连接。

2.1 SSE 超时短于 AI 超时

现象	影响	代码证据	建议方向
Scene emitter 超时 120 秒；AI 请求默认超时 180 秒。	模型仍在工作时 SSE 已关闭；后台请求继续运行，用户只看到断流。	SceneGenerationService.java:248；AiProperties.java:19-24	区分首 token、空闲和总超时；SSE 总超时大于上游；超时后取消模型请求。

2.2 阻塞流占用 JVM 公共线程池

现象	影响	代码证据	建议方向
CompletableFuture.runAsync 未指定执行器，内部执行同步 HttpClient.send 和持续 readLine。	并发流会长期占用公共 ForkJoinPool；没有容量、并发和拒绝策略。	SceneGenerationService.java:248-291；AiChatClient.java:135-188	使用独立有界执行器或异步 HTTP 流客户端，并暴露活动、排队和拒绝指标。

2.3 上游异常 EOF 可能被当成成功

现象	影响	代码证据	建议方向
JSON chunk 解析失败被静默忽略；未收到 [DONE] 的 EOF 仍正常返回并发送 done。	页面显示完成但内容缺失；协议变化或网络中断难以定位。	AiChatClient.java:171-197；SceneGenerationService.java:261-280	追踪 DONE/finish reason；非正常 EOF 判失败；chunk 解析失败计数并记录。

03 P0：项目进度 SSE 不可恢复

项目级 SSE 能展示实时事件，但不能保证事件送达。

ProgressEventPublisher 将 emitter 保存在本机内存，采用无限超时，但没有 heartbeat、事件 ID、Last-Event-ID、事件回放或多实例广播。前端断线后 EventSource 虽会自动重连，但断线期间发生的事件已经丢失。

缺失能力	直接后果	建议
Heartbeat	代理或网络设备清理空闲连接，服务端不及时发现	每 10-20 秒发送 comment/heartbeat
事件 ID	无法识别最后成功接收位置	所有事件带单调递增 ID
断线回放	scene.done、job.completed 可能永久丢失	近期事件缓存或最新状态快照
任务持久化	服务重启后只能看到粗粒度项目状态	MySQL job 表保存状态机
多实例广播	连接和任务不在同一实例时收不到事件	Redis Pub/Sub 或统一事件总线

当前代码位置

- backend/.../workflow/ProgressEventPublisher.java:19-132
- frontend/src/features/workbench/useWorkbench.ts:1001-1050

正确目标
SSE 不是必须做到每条事件强一致，但重连后最终状态必须正确。最低要求是立即发送持久化任务快照，而不是只依赖瞬时事件。

验收场景

- 任务运行中断网 10 秒再恢复，页面最终显示正确状态。
- 服务端空闲连接能够维持，或断开后及时清理 emitter。
- 多标签页订阅同一项目时不会互相影响。

04 P1：模型上下文和重复调用

这是降低首 token 时间、总耗时和费用的最高收益点。

4.1 每个 Scene 读取并发送全项目上下文

现象	影响	代码证据	建议方向
单 Scene 和流式预览都读取全部实体、全部故事事件，并完整写入 Prompt。	项目越大，首 token 越慢、费用越高、上下文越容易溢出；无关内容还会干扰生成。	SceneGenerationService.java:301-313、379-393、805-885	按 sourceRefs 过滤事件，按 characters/locationId 过滤实体，只提供必要邻接上下文。

4.2 流式预览与正式 Scene 重复调用模型

现象	影响	代码证据	建议方向
预览生成自由文本且不落库；正式生成再次请求 JSON。	同一 Scene 产生两次成本；预览和最终结果可能不同；预览不能参与最终校验。	streamScenePreview 与 generateSceneScript 两条独立链路	优先评估将流式链路升级为正式生成：服务端累积、结束后解析校验并持久化。

规模关系

当前模型输入成本近似为：

Scene 数 × 全项目实体和事件上下文

建议首先补充的指标

指标	用途
每次请求输入 token	验证上下文过滤是否真正降本
首 token P50/P95	衡量流式体感
单 Scene 总耗时	区分网络、模型和持久化耗时
结构/语义回归结果	防止为了减少上下文损害质量

05 P1：数据库和任务队列

数据库索引基本覆盖主要查询，当前问题更多是查询方式和全局串行。

5.1 Scene 批量生成存在 N+1 查询

现象	影响	代码证据	建议方向
每个 Scene 查询是否存在；生成时重复查大纲、项目、全实体、全事件；保存后再查结果。	SQL 数量随 Scene 数高倍增长；重复读取同一项目上下文。	SceneGenerationService.java:215-241、340-399	批次预加载大纲、已生成 ID、项目、实体和事件；构建上下文索引；减少写后查询。

5.2 RabbitMQ 默认全局串行

现象	影响	代码证据	建议方向
消费者 concurrency 和 max-concurrency 默认均为 1。	一个项目的长任务阻塞所有其他项目，形成队头阻塞。	application.yml:25-34；WorkflowJobService.java:145-202	先完成幂等和跨实例互斥，再允许不同项目有限并行；按任务成本分队列。

5.3 章节摘要逐章串行

现象	影响	代码证据	建议方向
每章同步调用模型并单独更新数据库，整个流程持有项目锁。	章节越多耗时线性累加；同项目其他操作长时间等待。	ChapterSummaryService.java:42-70	跳过未变化摘要；有限并行；批量更新；大任务放入 MQ。

不要先做的事
当前不建议先盲目扩大 Hikari 连接池。AI 等待和重复上下文才是主要耗时，扩大连接池不能解决流式断线和模型延迟。

06 P1：前端渲染和重复刷新

流式 chunk 的到达频率远高于页面适合重渲染的频率。

6.1 每个 chunk 重渲染整个工作台

现象	影响	代码证据	建议方向
每个 chunk 调用 setSceneStreamContent；大型 useWorkbench 状态驱动 DirectorCommandRoom 整体更新。	Three.js 地图和大量面板跟随流式 token 重渲染，低性能设备容易卡顿。	useWorkbench.ts:694-701；DirectorCommandRoom.tsx:33；ProductionMap.tsx:394	chunk 写 ref 后按 30-50ms 批量刷新；流式区独立状态；ProductionMap memo 和稳定 props。

6.2 完成事件导致重复数据刷新

现象	影响	代码证据	建议方向
业务 Service 和 RabbitMQ 外层都发 job.completed；outline.ready 也触发完整刷新。	重复读取实体、事件、大纲、项目、章节和项目列表，并可能重复推进下一任务。	WorkflowJobService.java:176-187；useWorkbench.ts:328-381、1022-1031	一个 job 只发布一个终态；阶段事件不驱动全量刷新；前端按事件类型局部刷新并去重。

前端验收指标

指标	目标
流式期间 ProductionMap 重渲染	不随每个 chunk 重渲染
chunk 到 UI 延迟	节流后仍保持自然的实时感
主线程长任务	流式期间无明显连续长任务
任务完成后的 API 请求数	无重复全量刷新

07 P2：状态与数据一致性

这些问题在服务重启、多实例或异常退出时放大。

问题	当前实现	风险	目标方案
项目锁	projectId → ReentrantLock 永久保存在本机内存	项目增加导致锁累积；多实例无效	可清理本地锁或数据库/Redis 分布式互斥
任务状态	ConcurrentHashMap 保存，无 TTL、无持久化	重启丢失、长期增长、无法审计	MySQL job 状态机；Redis 只承担短期状态/广播
全量分析	AI 调用前删除旧实体、事件、大纲和 Scene	异常退出后项目可能为空	新版本生成并校验后原子切换
Scene 重生成	先删除旧 Scene，再调用模型	崩溃时丢失旧版本	生成成功后 upsert，保留历史版本
MQ 失败治理	自动重试，无完整死信与业务幂等状态 机	部分落库后重复执行；失败难重放	幂等 job、DLQ、失败分类、人工重放

当前应避免的错误表述

- 不要说 Redis 已经解决任务状态、分布式锁或 SSE 广播；当前核心代码尚未使用它。
- 不要说 RabbitMQ 已经实现项目级并行；默认配置是所有项目共用一个串行消费者。
- 不要说 Scene 重生成天然安全；旧结果会先被删除。
- 不要把结构校验等同于文学或剧情质量评估。

08 建议的三阶段优化计划

每一阶段都必须有明确完成定义，避免同时改动所有链路。

阶段	范围	完成定义
阶段一 流式止血	关闭旧流；缓存状态化；统一超时；异常 EOF 判失败；heartbeat；专用执行器	不串流、不假成功、超时可清理、失败可重试
阶段二 延迟与成本	过滤 Scene 上下文；流式 UI 节流；地图隔离；消除 N+1；去重刷新	首 token 和 token 数下降；UI 流畅；SQL 和 API 请求下降
阶段三 任务生产化	job 表；有限并行；同项目幂等；事件恢复；多实例广播；DLQ；结果版本	重启可恢复；项目互不阻塞；重复消息不重复写；失败可审计

推荐开发顺序

- 1. 前端 EventSource 生命周期和缓存修复。
- 2. 服务端超时、正常完成判断、heartbeat 和清理。
- 3. 专用有界执行器或异步流客户端。
- 4. Scene 最小上下文与 token 指标。
- 5. 流式正式化或明确保留独立预览。
- 6. 前端节流、组件隔离和刷新去重。
- 7. 批量预加载、减少 N+1。
- 8. 任务持久化、有限并行和分布式一致性。

决策前置
团队需要先决定流式预览是继续作为独立功能，还是成为正式 Scene 生成通道。这个选择会影响接口协议、持久化、缓存和前端状态设计。

09 验证指标与测试清单

没有测量数据时，不能判断优化是否真正有效。

最小观测指标

维度	指标
流式	连接耗时、TTFT、chunk 间隔 P95、正常完成率、超时率、异常 EOF、解析失败、活动/排队流
AI	模型、阶段、输入/输出 token、耗时、重试、状态码、AI/兜底/解析失败
任务	MQ 等待、执行时间、重试次数、队列深度、最老消息等待时间
数据库	单 Scene SQL 数、查询耗时、事务耗时
前端	chunk 到显示延迟、主线程长任务、工作台和地图重渲染次数

必须覆盖的测试场景

- □ 流式中切换 Scene、切换项目、卸载页面。
- □ 首 token 前超时；返回部分 chunk 后断开；非法 chunk；正常 [DONE]。
- □ 项目 SSE 断网后重连；多个标签页订阅同一项目。
- □ 多个项目同时执行；同一任务重复提交和重复消费。
- □ 任务中途重启服务；重生成失败时旧 Scene 仍可用。
- □ 中型项目测量 Prompt token、SQL 数、TTFT 和重渲染次数。

日志边界
可记录 projectId、sceneId、模型、阶段、token、耗时和状态码；不得记录密钥、完整小说原文、完整 Prompt 或完整模型响应。

结论 当前最主要的问题

供团队会议直接引用。

第一结论

流式不可靠首先是生命周期和协议完成判断问题：旧流不关闭、失败缓存不区分、超时互相冲突、异常 EOF 会假成功。

第二结论

当前最大的性能和成本浪费是每个 Scene 重复携带全项目上下文，以及预览和正式生成重复调用模型。

第三结论

RabbitMQ、SSE

和项目锁已经建立了工程骨架，但任务状态仍是本机内存、消费者默认全局串行，尚未达到多实例生产状态。

建议下一步

先单独完成“流式可靠性止血”设计和验证，不同时重构整个任务系统。该阶段完成后，再用 TTFT、正常完成率、输入 token、SQL 数和前端重渲染数据决定第二阶段优化范围。

依据文档：docs/performance-reliability-review.md。本文为静态代码审查结论，尚未实施业务修改，也不替代后续压测和故障注入结果。